

INDEX

Serial No.	Experiment Name	Page No.
01	Write a Python program to convert a given text to lowercase and remove all punctuation.	2
02	Write a Python program to tokenize a paragraph into sentences and words using NLTK.	4
03	Write a Python program to remove English stopwords from tokenized words using NLTK.	6
04	Write a Python program to apply stemming to words using NLTK's PorterStemmer.	8
05	Write a Python program to apply lemmatization to words using NLTK's WordNetLemmatizer.	10
06	Write a Python program to perform Part-of-Speech (POS) tagging on a sentence using NLTK.	12
07	Write a Python program to show the most frequent words in a text using NLTK FreqDist.	14
08	Write a Python program to generate bigrams (2-word sequences) from a text using NLTK.	16
09	Write a Python program to perform sentiment analysis (Positive/Negative/Neutral) using NLTK's VADER.	18
10	Named Entity Recognition (NER) (Slightly Moderate) Write a Python program to perform basic Named Entity Recognition (NER) and chunking using NLTK.	20

Write a Python program to convert a given text to lowercase and remove all punctuation.

Objective

- To understand the concept of text cleaning in Natural Language Processing.
- To implement text cleaning using Python and NLTK libraries.

Theory

Text cleaning is an essential preprocessing step in many natural language processing (NLP) tasks. In this experiment, we will perform two common text cleaning operations:

1. **Lowercasing:** This involves converting all characters in a string to lowercase. This helps in making the text case-insensitive, ensuring that words like "Python" and "python" are treated the same.
2. **Punctuation Removal:** This step involves removing punctuation marks like commas, periods, exclamation marks, etc., to focus only on the textual content.

Python provides several methods to handle text. The lower() method can be used to convert text to lowercase, and we can use the str.translate() method with a translation table to remove punctuation.

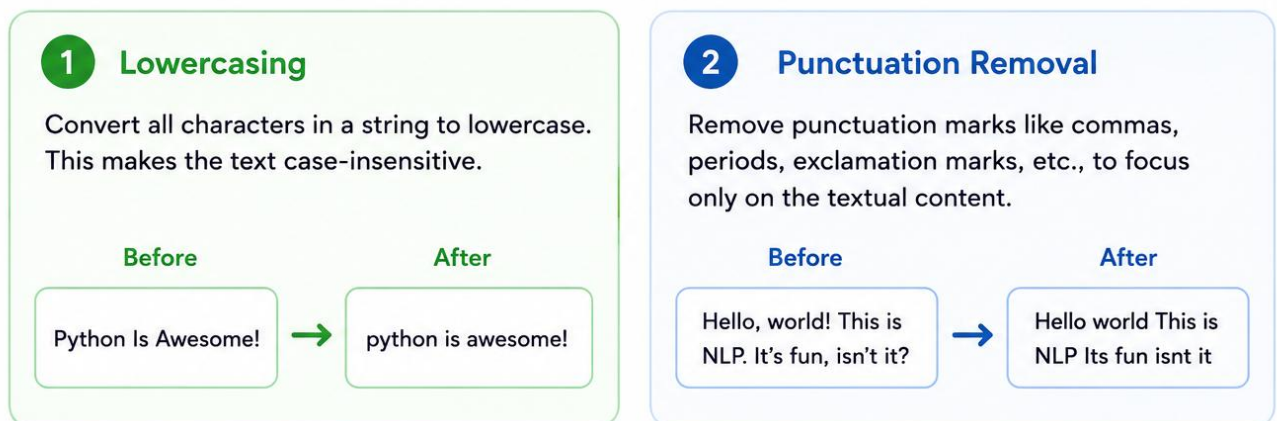


Figure-1: NLP Text cleaning

Python Code

```
import string

text = "Hello, World! This is Natural Language Processing."

cleaned = text.lower().translate(str.maketrans('', '',
string.punctuation))

print(cleaned)
```

Example Input & Output



Example Input

"Hello, World! This is Natural
Language Processing."



Example Output

hello world this is natural
language processing

Write a Python program to tokenize a paragraph into sentences and words using NLTK.

Objective

The objective of this experiment is to write a Python program that tokenizes a given paragraph into sentences and words using the Natural Language Toolkit (NLTK) library.

Theory

Tokenization is a fundamental step in the field of Natural Language Processing (NLP). It is the process of splitting a string of text into smaller, manageable pieces called tokens. These tokens can be words, sentences, or even characters, depending on the type of tokenization applied.

There are two primary types of tokenization commonly used:

1. **Sentence Tokenization:** This type of tokenization splits a paragraph or document into individual sentences. The sentence tokenizer looks for punctuation marks like periods (.), exclamation marks (!), and question marks (?) to identify where each sentence ends. The purpose of sentence tokenization is to break down a large chunk of text into smaller units, making it easier to analyze sentence-level structure or perform sentence-specific processing.
2. **Word Tokenization:** In word tokenization, the text is split into individual words. This is done by identifying spaces between words and treating punctuation marks, such as commas (,), periods (.), and quotation marks (" ") as separate tokens. Word tokenization allows for a deeper analysis of the structure of sentences and is essential for tasks like part-of-speech tagging, sentiment analysis, and text classification. In this experiment, we will use the NLTK (Natural Language Toolkit) library, which provides built-in functions for tokenization. NLTK's `sent_tokenize` and `word_tokenize` functions help break down text into sentences and words, respectively, making it easier for NLP tasks to operate on the text.



Figure-2: Tokenization in NLP Using NLTK

Python Code

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
nltk.download('punkt', quiet=True)

text = "Hello world. This is an NLP course. It is very interesting!"

sentences = sent_tokenize(text)
words      = word_tokenize(text)

print("Sentences:", sentences)
print("Words:", words)
```

Example Input & Output



Example Input

```
"Hello world. This is an NLP
course. It is very interesting!"
```



Example Output

```
Sentences: ['Hello world.', 'This
is an NLP course.',
            'It is very
interesting!']
Words: ['Hello', 'world', '.',
        'This', 'is', 'an',
        'NLP', 'course', '.', 'It',
        'is', 'very',
        'interesting', '!']
```

Write a Python program to remove English stopwords from tokenized words using NLTK.

Objective

The objective of this experiment is to write a Python program to remove English stopwords from tokenized words using the Natural Language Toolkit (NLTK)

Theory

In Natural Language Processing (NLP), stopwords refer to common words that are often removed from a text before analysis. These words, such as "the", "is", "in", "on", "and", and "a", do not contribute significant meaning to the text in most cases and are often considered as noise. Removing stopwords is a key step in text preprocessing because it helps reduce the size of the data and makes the analysis more focused on meaningful content.

In this experiment, we will use the NLTK library to remove English stopwords from a list of tokenized words. NLTK provides a predefined list of stopwords in multiple languages, including English. The program will:

1. Tokenize a given sentence or paragraph into words.
2. Compare each word against the NLTK stopwords list.
3. Remove any word that is found in the stopwords list.

By removing these unnecessary words, the text becomes more suitable for downstream tasks like text classification, sentiment analysis, and information retrieval.

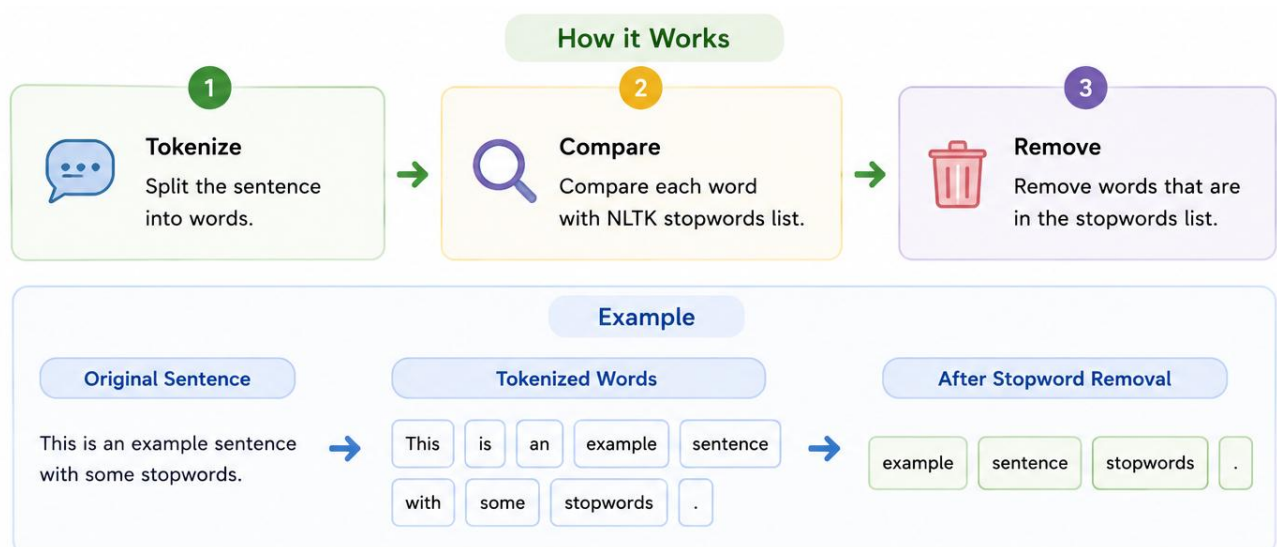


Figure-3: Stopword removal using NLTK.

Python Code

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('stopwords', quiet=True)

text = "This is a simple sentence for removing stopwords in NLP."
words = word_tokenize(text)

stop_words = set(stopwords.words('english'))
filtered = [w for w in words if w.lower() not in stop_words]

print(filtered)
```

Example Input & Output



Example Input

"This is a simple sentence for removing stopwords in NLP."



Example Output

['simple', 'sentence', 'removing', 'stopwords', 'NLP', '.']

Write a Python program to apply stemming to words using NLTK's PorterStemmer.

Objective

The objective of this experiment is to write a Python program that applies stemming to words using NLTK's PorterStemmer.

Theory

Stemming is a text normalization technique in Natural Language Processing (NLP) where words are reduced to their root or base form. This process helps to simplify words with different tenses, derivations, or forms into a single form, often known as the stem. For example:

- "running" → "run"
- "better" → "better" (no change, as "better" is already the base form)
- "happily" → "happi"

Stemming is different from lemmatization. While stemming simply chops off prefixes and suffixes to reduce words to their root form, lemmatization uses vocabulary and morphological analysis to obtain the base form of the word (i.e., a valid dictionary word).

In this experiment, we will use the Porter Stemmer, which is one of the most widely used stemming algorithms in NLP. It applies a set of rules to remove common suffixes from words.

The steps in stemming involve:

1. Tokenizing the input text into words.
2. Applying the PorterStemmer algorithm to each word.
3. Outputting the stemmed words.

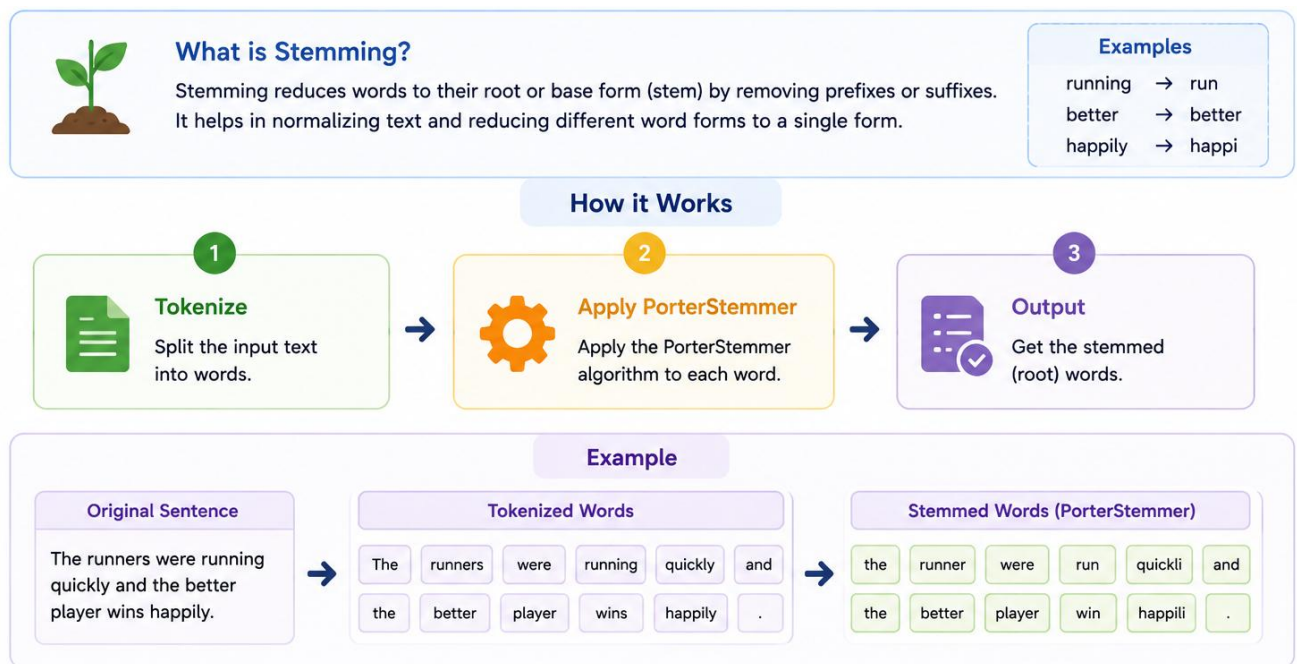


Figure-4: Stemming using NLTK porterstemmer.

Python Code

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt', quiet=True)

ps = PorterStemmer()
text = "The cats are running and jumping quickly."
words = word_tokenize(text)

stemmed = [ps.stem(w) for w in words]
print(stemmed)
```

Example Input & Output



Example Input

"The cats are running and jumping quickly."



Example Output

['the', 'cat', 'are', 'run', 'and', 'jump', 'quickli', '.']

Write a Python program to apply lemmatization to words using NLTK's WordNetLemmatizer.

Objective

The objective of this experiment is to write a Python program that applies lemmatization to words using NLTK's WordNetLemmatizer.

Theory

Lemmatization is a more advanced form of text normalization compared to stemming. While stemming only removes common prefixes or suffixes, lemmatization involves reducing a word to its base or dictionary form, called a lemma, by considering its meaning and part of speech (POS). This results in words that are valid words in the dictionary.

For example:

- "running" → "run" (verb)
- "better" → "good" (adjective)
- "mice" → "mouse" (noun)

Lemmatization generally provides more accurate and meaningful results compared to stemming because it takes into account the word's context (such as whether the word is a noun, verb, etc.). The WordNetLemmatizer in NLTK uses the WordNet lexical database to map words to their correct lemma based on the context.

In this experiment, we will:

1. Tokenize the input text into words.
2. Apply WordNetLemmatizer to each word to find its lemma.
3. Output the lemmatized words.

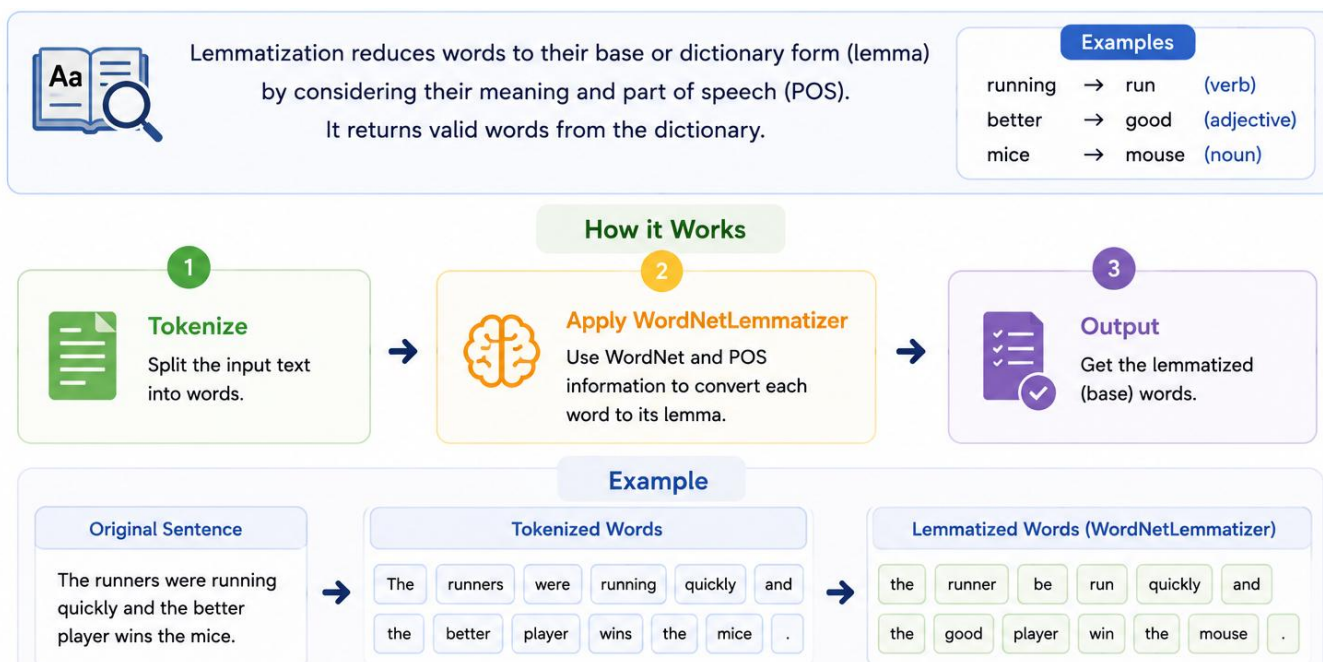


Figure-5: Lemmatization using NLTK's WordNetLemmatizer

Python Code

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
nltk.download('wordnet', quiet=True)

lemmatizer = WordNetLemmatizer()
text = "The cats are running and jumping quickly."
words = word_tokenize(text)

lemmatized = [lemmatizer.lemmatize(w) for w in words]
print(lemmatized)
```

Example Input & Output

Example Input

"The cats are running and jumping quickly."

Example Output

['The', 'cat', 'are', 'running', 'and', 'jumping', 'quickly', '.']

Write a Python program to perform Part-of-Speech (POS) tagging on a sentence using NLTK.

Objective

The objective of this experiment is to write a Python program to perform Part-of-Speech (POS) tagging on a given sentence using NLTK.

Theory

Part-of-Speech (POS) tagging is a process in Natural Language Processing (NLP) where each word in a sentence is assigned a specific part of speech (POS), such as noun, verb, adjective, adverb, etc. POS tagging is essential in understanding the grammatical structure of a sentence and plays a crucial role in many NLP tasks, such as named entity recognition, text classification, and syntactic parsing.

NLTK (Natural Language Toolkit) provides an easy way to perform POS tagging through its built-in `pos_tag()` function. The POS tags in NLTK are based on the Penn Treebank tag set, which includes the following major categories:

- NN: Noun (singular)
- NNS: Noun (plural)
- VB: Verb (base form)
- VBD: Verb (past tense)
- JJ: Adjective
- RB: Adverb
- PRP: Pronoun
- DT: Determiner (e.g., "the", "a")

In this experiment, we will:

1. Tokenize the input sentence into words.
2. Use NLTK's `pos_tag()` function to tag each word with its corresponding part of speech.
3. Output the tagged words and their corresponding POS tags.

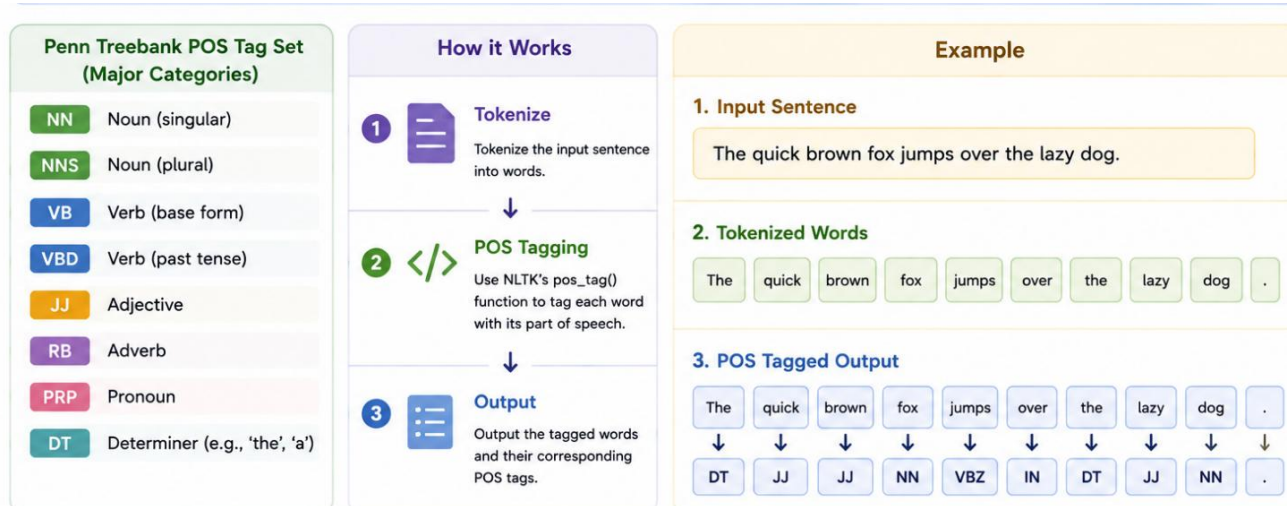


Figure-6: Part-of-Speech (POS) Tagging using NLTK

Python Code

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('averaged_perceptron_tagger', quiet=True)

text = "The quick brown fox jumps over the lazy dog."
words = word_tokenize(text)

pos_tags = nltk.pos_tag(words)
print(pos_tags)
```

Example Input & Output



Example Input

"The quick brown fox jumps over the lazy dog."



Example Output

```
[('The', 'DT'), ('quick', 'JJ'), ('brown', 'JJ'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('.', '.')] ]
```

Write a Python program to show the most frequent words in a text using NLTK FreqDist.

Objective

The objective of this experiment is to write a Python program that shows the most frequent words in a given text using NLTK's FreqDist (Frequency Distribution).

Theory

Frequency Distribution is an important concept in Natural Language Processing (NLP) used to measure how often words or tokens appear in a text. Words with higher frequency are considered more important and useful for tasks like text summarization, keyword extraction, and sentiment analysis.

NLTK provides the FreqDist() class to calculate and display word frequencies easily. The process involves the following steps:

1. **Text Tokenization:**

The input text is first divided into smaller units called tokens using NLTK's word_tokenize() function.

2. **Frequency Distribution:**

The FreqDist() class counts how many times each word appears in the tokenized text and stores the frequencies.

3. **Displaying Common Words:**

Using the most_common(n) method, we can identify the most frequent words in the text.

4. **Visualization:**

NLTK also provides the plot() method to display word frequencies graphically using a histogram.

The FreqDist class helps to:

- I. Break text into tokens.
- II. Count word frequencies.
- III. Identify the most common words.
- IV. Visualize frequency distribution.



OVERVIEW

Frequency Distribution is used to find how often each word (or token) appears in a text. It helps identify important or prominent words and is useful in many NLP tasks like text summarization, keyword extraction, topic modeling, and sentiment analysis.

NLTK TOOL

NLTK's `FreqDist()` class provides an easy way to calculate and display word frequencies.



1 Text Tokenization

The input text is split into individual words (tokens) using NLTK's `word_tokenize()` function.

Example Text

NLTK is a powerful library for natural language processing. NLTK makes NLP easy and fun!

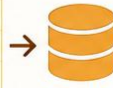
Tokenized Words

['NLTK', 'is', 'a', 'powerful', 'library', 'for', 'natural', 'language', 'processing', 'NLTK', 'makes', 'NLP', 'easy', 'and', 'fun', '!']

2 Frequency Distribution

`FreqDist()` counts how many times each word appears and stores the frequencies in a dictionary-like structure.

Word	Frequency
nltk	2
is	1
a	1
powerful	1
library	1
...	...



FreqDist (Object)

3 Displaying Common Words

The `most_common(n)` method returns the top N most frequent words, sorted by frequency.

Top 5 Most Common Words (`most_common(5)`)

Word	Frequency
nltk	2
is	1
a	1
powerful	1
library	1

4 Visualizing Word Frequency

The `plot()` method creates a histogram of the most common words.

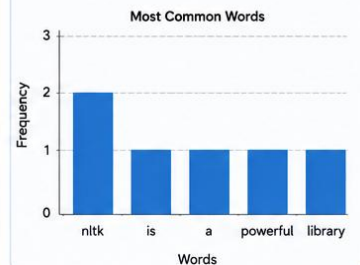


Figure-7: Frequency Distribution using NLTK `FreqDist()`

Python Code

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist
nltk.download('punkt', quiet=True)

text = "NLP is fun. NLP is interesting. NLP helps in many applications."
words = word_tokenize(text)

fdist = FreqDist(words)
print(fdist.most_common(5))
```

Example Input & Output



Example Input

"NLP is fun. NLP is interesting.
NLP helps in many applications."



Example Output

[('NLP', 3), ('.', 2), ('is', 2), ('fun', 1), ('interesting', 1)]

Write a Python program to generate bigrams (2-word sequences) from a text using NLTK.

Objective

The objective of this experiment is to write a Python program that generates bigrams (2-word sequences) from a given text using NLTK.

Theory

N-grams are sequences of n words or tokens from a text. In Natural Language Processing (NLP), n-grams are widely used for text analysis, language modeling, machine translation, and information retrieval. A **bigram** is a sequence of two consecutive words ($n = 2$).

For example, for the sentence "**The cat sleeps**", the bigrams are:

- ('The', 'cat')
- ('cat', 'sleeps')

Concept of N-grams:

An n-gram model breaks text into smaller chunks to capture relationships between adjacent words. These include:

- **Unigrams:** ['The', 'cat', 'sleeps']
- **Bigrams:** [('The', 'cat'), ('cat', 'sleeps')]
- **Trigrams:** [('The', 'cat', 'sleeps')]

Bigram generation focuses on pairs of neighboring words. For example:

- "Natural language processing" → [('Natural', 'language'), ('language', 'processing')]

Creating Bigrams:

Generating bigrams usually involves two steps:

1. **Tokenization:**
Splitting text into individual words or tokens.
2. **Bigram Generation:**
Pairing each word with the next word in the sequence.

Example:

- Text: "Natural language processing enables machines."
- Tokenized: ['Natural', 'language', 'processing', 'enables', 'machines']
- Bigrams: [('Natural', 'language'), ('language', 'processing'), ('processing', 'enables'), ('enables', 'machines')]

NLTK's bigrams() Function:

NLTK provides the `bigrams()` function to generate bigrams easily. It takes tokenized words as input and returns pairs of consecutive words.

How it works:

- Receives tokenized text as input.
- Creates adjacent word pairs.
- Returns the output as tuples of bigrams.

Example:

- Sentence: "I love Python programming."
- Tokenized: ['I', 'love', 'Python', 'programming']
- Bigrams: [('I', 'love'), ('love', 'Python'), ('Python', 'programming')]

Example of Bigram Generation:

Sentence: "The quick brown fox jumps over the lazy dog."

- Tokenized words: ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog']
- Bigrams generated: [('The', 'quick'), ('quick', 'brown'), ('brown', 'fox'), ('fox', 'jumps'), ('jumps', 'over'), ('over', 'the'), ('the', 'lazy'), ('lazy', 'dog')]

These bigrams are useful for NLP tasks like frequency analysis, sentiment detection, and text generation.

Python Code

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
nltk.download('punkt', quiet=True)

text = "The quick brown fox jumps over the lazy dog."
words = word_tokenize(text)

bigrams_list = list(ngrams(words, 2))
print(bigrams_list)
```

Example Input & Output



Example Input

"The quick brown fox jumps over the lazy dog."



Example Output

```
[('The', 'quick'),
 ('quick', 'brown'), ('brown', 'fox'),
 ('fox', 'jumps'), ('jumps', 'over'),
 ('over', 'the'),
 ('the', 'lazy'), ('lazy', 'dog'),
 ('dog', '.')] ]
```

Write a Python program to perform sentiment analysis (Positive/Negative/Neutral) using NLTK's VADER.

Objective

The objective of this experiment is to perform sentiment analysis on a given text and categorize it into Positive, Negative, or Neutral using NLTK's VADER sentiment analysis tool.

Theory

Sentiment Analysis is a task in Natural Language Processing (NLP) that determines the sentiment or opinion expressed in a piece of text. It classifies text into categories such as positive, negative, or neutral based on the words and phrases used.

There are different approaches to sentiment analysis in NLP, including:

1. **Lexicon-based Approaches:**

These approaches use predefined dictionaries (lexicons) where words are associated with sentiment scores.

2. **Machine Learning-based Approaches:**

These approaches use labeled training data to learn and predict sentiment from text features.

VADER Sentiment Analysis

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a lexicon and rule-based sentiment analysis tool designed mainly for social media text. It is included in the NLTK library and provides an easy way to classify text into sentiment categories.

VADER works by assigning sentiment scores to words and applying rules that consider punctuation, capitalization, and negation.

Sentiment Categories in VADER:

- **Positive Sentiment:** The text contains more positive words.
- **Negative Sentiment:** The text contains more negative words.
- **Neutral Sentiment:** The text has no strong emotional meaning.
- **Compound Sentiment Score:** A normalized score that combines all sentiment values.

The compound score ranges from **-1** (most negative) to **+1** (most positive). A score close to **0** indicates neutral sentiment.

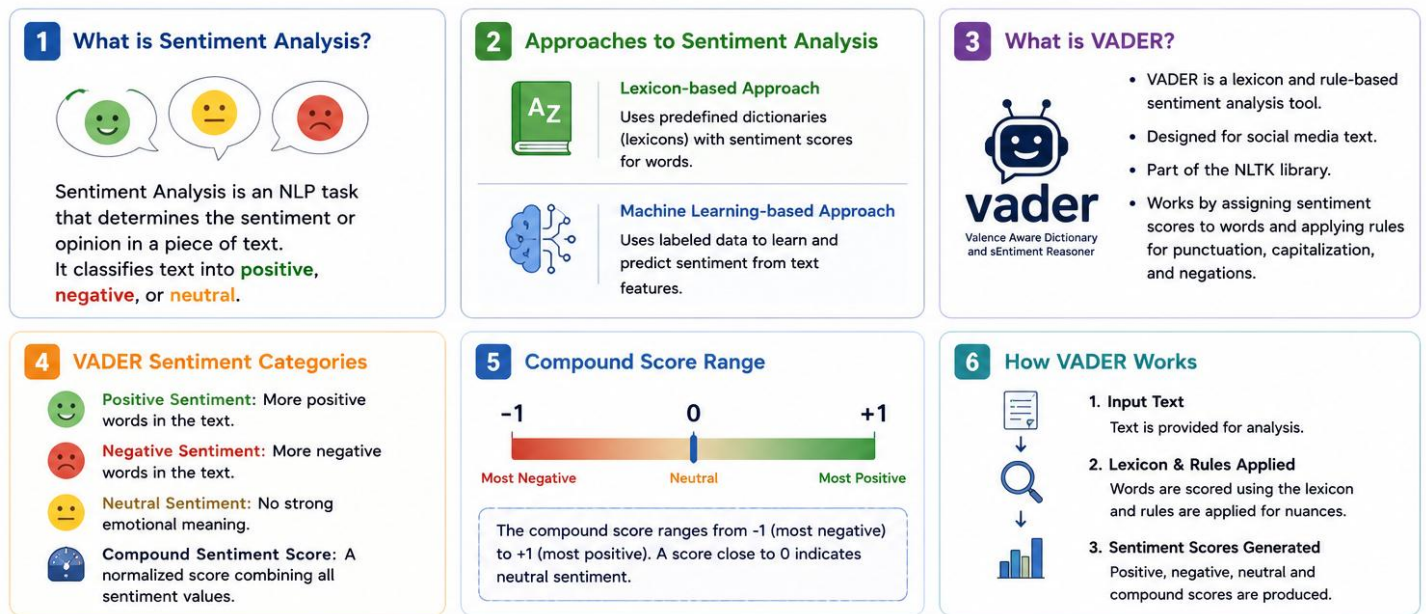


Figure-9: Sentiment Analysis using VADER in NLTK

Python Code

```
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
nltk.download('vader_lexicon', quiet=True)

sia = SentimentIntensityAnalyzer()
text = "I love this NLP course. It is amazing and very useful!"

scores = sia.polarity_scores(text)
compound = scores['compound']

if compound >= 0.05:
    sentiment = "Positive"
elif compound <= -0.05:
    sentiment = "Negative"
else:
    sentiment = "Neutral"

print("Scores:", scores)
print("Overall Sentiment:", sentiment)
```

Example Input & Output



Example Input

"I love this NLP course. It is amazing and very useful!"



Example Output

Scores: {'neg': 0.0, 'neu': 0.318, 'pos': 0.682, 'compound': 0.7845}
Overall Sentiment: Positive

EXP

10

Experiment Name

Named Entity Recognition (NER) (Slightly Moderate)

Write a Python program to perform basic Named Entity Recognition (NER) and chunking using NLTK.

Objective

The objective of this experiment is to perform Named Entity Recognition (NER) and chunking on a given text using the NLTK (Natural Language Toolkit) library, focusing on identifying named entities and creating syntactically meaningful chunks in the text.

Theory

Named Entity Recognition (NER) and Chunking in NLTK

Named Entity Recognition (NER) is an important task in Natural Language Processing (NLP) used to identify and classify named entities in text. These entities may include:

- Persons (e.g., "Barack Obama")
- Organizations (e.g., "Google")
- Locations (e.g., "Paris")
- Dates and monetary values

NER can be performed using:

1. **Rule-based NER** – Uses predefined rules and dictionaries.
2. **Machine Learning-based NER** – Uses trained models to classify entities from text features.

In NLTK, the `ne_chunk()` function is used for NER. It labels entities such as PERSON, ORGANIZATION, and GPE (Geopolitical Entity).

Chunking

Chunking is the process of grouping words into meaningful phrases, such as:

- Noun Phrases (NP)
- Verb Phrases (VP)
- Prepositional Phrases (PP)

Chunking is usually done after POS tagging. NLTK uses `RegexpParser` and grammar rules for chunking. Example:

NP: {<DT>?<JJ>*<NN.*>}

This rule identifies noun phrases with an optional determiner, adjectives, and nouns.

NLTK Tools for NER and Chunking

- `pos_tag()` – Assigns POS tags to words.
- `ne_chunk()` – Performs Named Entity Recognition.
- `RegexpParser` – Creates chunks using grammar rules.

Combining NER and Chunking

NER identifies important entities, while chunking groups words into meaningful structures.

Example:

“Barack Obama was born in Honolulu, Hawaii.”

- NER identifies “Barack Obama” as PERSON and “Honolulu” as GPE.
- Chunking identifies phrases like “Barack Obama” and “Honolulu, Hawaii” as noun phrases.

Python Code

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag, ne_chunk
nltk.download('maxent_ne_chunker', quiet=True)
nltk.download('words', quiet=True)

text = "Barack Obama was born in Hawaii and worked at Microsoft."
words = word_tokenize(text)
pos_tags = pos_tag(words)

ner_tree = ne_chunk(pos_tags)
print(ner_tree)
```

Example Input & Output



Example Input

"Barack Obama was born in Hawaii and worked at Microsoft."



Example Output

```
(S
 (PERSON Barack/NNP)
 (PERSON Obama/NNP)
 was/VBD born/VBN in/IN
 (GPE Hawaii/NNP)
 and/CC worked/VBD at/IN
 (ORGANIZATION Microsoft/NNP)
 ./.)
```